

LAPORAN UAS
PEMROGRAMAN BERORIENTASI OBJEK



Disusun Oleh :

- 1. Nazia Tasnim (25104410005)**
- 2. Silki Dawaain Nisa' (25104410006)**
- 3. Inayatu Nafisah (25104410008)**
- 4. M Nashril Aziz Al-Azhar (25104410013)**
- 5. Zafisca Hidayatul Ula (25104410019)**
- 6. Ahmad Shofa Hasanul B. (25104410024)**

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK DAN INFORMATIKA
UNIVERSITAS ISLAM BALITAR
BLITAR
2026

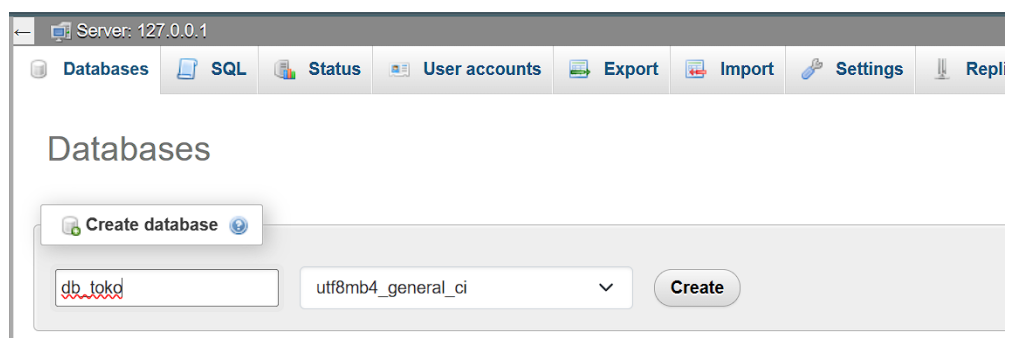
Aplikasi Pengelolaan Data Produk dan Kategori Berbasis Java

Database yang digunakan pada aplikasi ini adalah db_toko yang dibuat menggunakan MySQL. Database ini terdiri dari dua tabel utama, yaitu tabel kategori dan tabel produk. Tabel kategori digunakan untuk menyimpan data kategori barang, sedangkan tabel produk digunakan untuk menyimpan informasi produk seperti nama produk, merek (brand), harga, stok, dan kategori. Kedua tabel saling terhubung melalui Foreign Key, sehingga setiap produk harus memiliki kategori yang valid. Penerapan Foreign Key juga bertujuan untuk menjaga konsistensi dan integritas data dalam database.

1. Implementasi Database

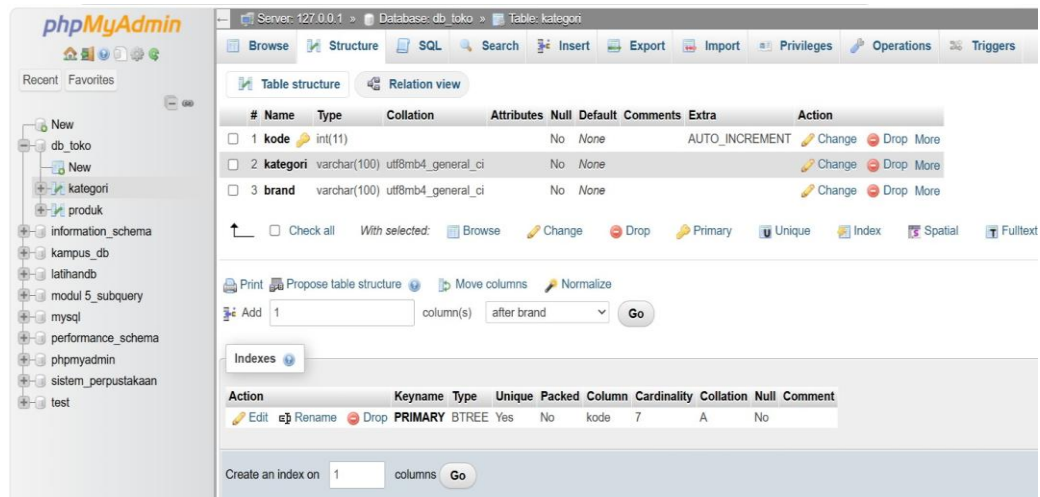
1.1. Pembuatan Database db_toko

Langkah pertama dalam pembuatan aplikasi adalah membuat database dengan nama db_toko menggunakan MySQL melalui phpMyAdmin. Database ini berfungsi sebagai tempat penyimpanan seluruh data yang digunakan oleh aplikasi, seperti data kategori dan data produk.



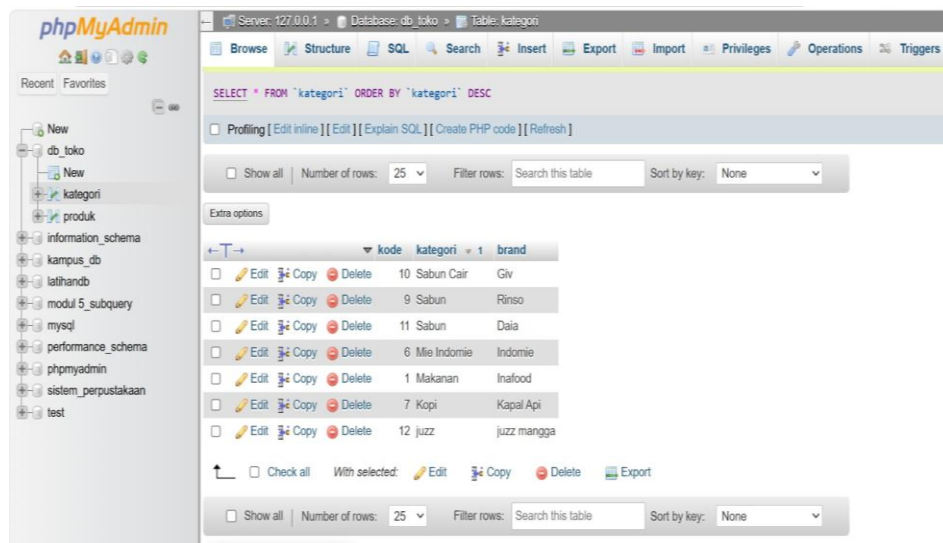
1.2. Pembuatan Tabel kategori

Setelah database berhasil dibuat, langkah berikutnya adalah membuat tabel kategori. Tabel ini digunakan untuk menyimpan data kategori produk yang nantinya akan digunakan sebagai acuan dalam pengelompokan produk.



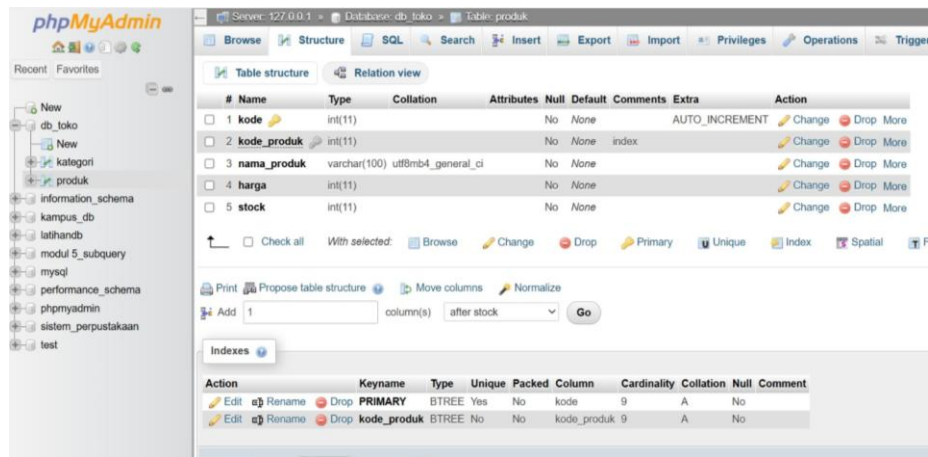
1.3. Tampilan Browse Tabel kategori

Setelah tabel kategori berhasil dibuat, tampilan Browse pada phpMyAdmin digunakan untuk menampilkan data yang tersimpan di dalam tabel. Pada tahap ini, tabel telah berhasil dibuat dan siap digunakan untuk menyimpan data kategori produk.



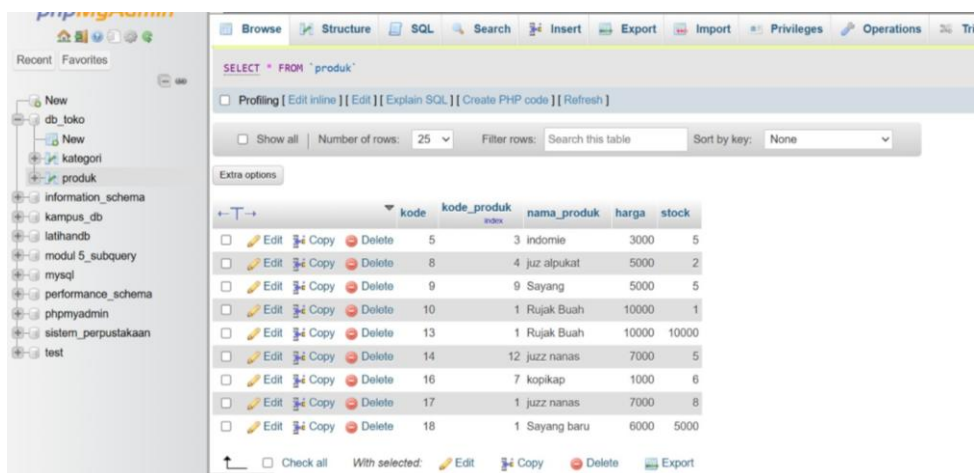
1.4. Pembuatan Tabel produk

Selanjutnya dibuat tabel produk yang digunakan untuk menyimpan informasi setiap produk, seperti nama produk, merek (brand), harga, stok, dan kategori. Tabel ini memiliki relasi dengan tabel kategori melalui Foreign Key sehingga setiap produk harus memiliki kategori yang valid.



1.5. Tampilan Browse Tabel produk

Gambar berikut menunjukkan tampilan Browse pada tabel produk. Tampilan ini berfungsi untuk menampilkan data produk yang telah tersimpan di dalam database dan memastikan bahwa tabel dapat digunakan dengan baik oleh aplikasi.



2. Implementasi Source Code

2.1 DBConnection.java

Class DBConection berfungsi sebagai penghubung antara aplikasi Java dengan database MySQL db_toko. Class ini menyediakan koneksi yang nantinya digunakan oleh class KategoriDAO dan ProdukDAO untuk melakukan operasi CRUD pada database.

```

18 public class DBConnection {
19
20     static final String DB_URL = "JDBC:mysql://localhost:3306/db_toko";
21     static final String DB_user = "root";
22     static final String DB_PASS = "";
23
24     public static void main(String[] args) {
25         Connection conn = getKoneksi(); //
26         if(conn != null){
27             try{
28                 conn.close();
29                 System.out.println("koneksi ditutup");
30             }
31             catch(SQLException e){
32                 System.err.println("ERROR SAAT MENUTUP KONEKSI");
33             }
34         }
35     }

```

Baris 20 - 23 dari Potongan source code di atas digunakan untuk menyimpan konfigurasi koneksi ke database. DB_URL berisi alamat database db_toko yang berada pada server lokal (localhost), sedangkan DB_user dan DB_PASS berisi username serta password yang digunakan untuk mengakses database MySQL.

```

37     public static Connection getKoneksi() { // return
38         Connection conn = null;
39         try { // satu paket dengan catch
40             conn = DriverManager.getConnection(DB_URL,DB_user,DB_PASS);
41             if(conn!=null){
42                 //disini ada koneksi
43                 System.out.println("koneksi berhasil");
44                 //
45             }
46             else {
47                 System.out.println("koneksi tidak berhasil");
48             }
49         }
50         catch(SQLException e){
51             System.err.println("Error koneksi ke database gagal"+e.getMessage());
52         }
53         return conn;
54     }
55 }

```

Method getKoneksi() digunakan untuk membuat koneksi antara aplikasi Java dengan database MySQL. Proses koneksi dilakukan menggunakan DriverManager.getConnection() dengan memanfaatkan URL database, username, dan password yang telah ditentukan sebelumnya. Apabila koneksi berhasil, program akan menampilkan pesan "koneksi berhasil", sedangkan jika terjadi kesalahan, pesan error akan ditampilkan melalui blok catch. Objek koneksi kemudian

dikembalikan menggunakan return conn agar dapat digunakan oleh class lain, seperti KategoriDAO dan ProdukDAO.

2.2 Kategori.java

Class Kategori merupakan model class yang digunakan untuk merepresentasikan data kategori pada aplikasi Sistem Kasir. Class ini berfungsi sebagai tempat penyimpanan data kategori yang terdiri dari kode, nama kategori, dan brand. Data yang disimpan pada class ini nantinya akan digunakan oleh class KategoriDAO untuk melakukan proses penyimpanan maupun pengambilan data dari database.

```
14 public class Kategori {
15
16     private int kode;
17     private String kategori;
18     private String brand;
19
20     public Kategori() {
21     }
22
23     public Kategori(int kode, String kategori, String brand) {
24         this.kode = kode;
25         this.kategori = kategori;
26         this.brand = brand;
27     }
28
29     public int getKode() {
30         return kode;
31     }
32
33     public void setKode(int kode) {
34         this.kode = kode;
35     }
36 }
```

Baris 16–18 pada potongan source code di atas merupakan deklarasi atribut pada class Kategori. Variabel kode digunakan untuk menyimpan kode kategori, variabel kategori digunakan untuk menyimpan nama kategori, sedangkan variabel brand digunakan untuk menyimpan nama merek. Ketiga atribut tersebut merupakan data utama yang dimiliki oleh objek Kategori.

Baris 23–26 merupakan constructor pada class Kategori. Constructor ini digunakan untuk menginisialisasi objek dengan memberikan nilai awal pada atribut kode, kategori, dan brand melalui parameter yang diterima. Dengan demikian, objek Kategori dapat langsung digunakan setelah dibuat.

```

37 | public String getKategori() {
38 |     return kategori;
39 | }
40 |
41 | public void setKategori(String kategori) {
42 |     this.kategori = kategori;
43 | }
44 |
45 | public String getBrand() {
46 |     return brand;
47 | }
48 |
49 | public void setBrand(String brand) {
50 |     this.brand = brand;
51 | }
52 |
53 | @Override
54 | public String toString() {
55 |     return kode + " - " + kategori + " - " + brand;
56 | }
57 | }

```

Potongan Sourcode diatas merupakan method getter dan setter. Method getter digunakan untuk mengambil nilai dari atribut, sedangkan method setter digunakan untuk mengubah nilai atribut. Penggunaan getter dan setter merupakan penerapan konsep enkapsulasi (encapsulation) dalam pemrograman berorientasi objek karena atribut yang bersifat private hanya dapat diakses melalui method tersebut.

2.3 Produk.java

Class Produk merupakan model class yang digunakan untuk merepresentasikan data produk pada aplikasi Sistem Kasir. Class ini berfungsi sebagai tempat penyimpanan data produk yang meliputi kode kategori, kode produk, nama produk, harga, dan stok. Data yang terdapat pada class ini akan digunakan oleh class ProdukDAO dalam proses pengolahan data, seperti menambah, mengubah, menghapus, dan menampilkan data produk dari database.

```

14 public class Produk {
15     private int kode;
16     private int kode_produk;
17     private String nama_produk;
18     private int harga;
19     private int stock;
20
21     public Produk() {
22     }
23
24     public Produk(int kode, int kodeProduk, String nama, int harga, int stock) {
25         this.kode = kode;
26         this.kode_produk = kodeProduk;
27         this.nama_produk = nama;
28         this.harga = harga;
29         this.stock = stock;
30     }
31
32     public int getKode() {
33         return kode;
34     }
35
36     public void setKode(int kode) {
37         this.kode = kode;
38     }

```

Baris 15 – 19 dari Potongan source code di atas merupakan deklarasi atribut pada class Produk. Variabel kode digunakan untuk menyimpan kode kategori, kode_produk digunakan sebagai identitas setiap produk, nama_produk menyimpan nama produk, harga menyimpan harga produk, sedangkan stock digunakan untuk menyimpan jumlah stok produk yang tersedia.

Baris 24 – 30 dari potongan sourcode di atas merupakan Constructor yang digunakan untuk menginisialisasi objek Produk. Saat objek dibuat, nilai kode kategori, kode produk, nama produk, harga, dan stok akan langsung diisi melalui parameter yang diberikan sehingga objek siap digunakan dalam proses pengolahan data.

```

40     public int getKodeProduk() {
41         return kode_produk;
42     }
43
44     public void setKodeProduk(int kodeProduk) {
45         this.kode_produk = kodeProduk;
46     }
47
48     public String getNama() {
49         return nama_produk;
50     }
51
52     public void setNama(String nama) {
53         this.nama_produk = nama;
54     }
55
56     public int getHarga() {
57         return harga;
58     }
59
60     public void setHarga(int harga) {
61         this.harga = harga;
62     }

```



```

64     public int getStock() {
65         return stock;
66     }
67
68     public void setStock(int stock) {
69         this.stock = stock;
70     }
71 }

```

Method getter dan setter digunakan untuk mengambil serta mengubah nilai atribut yang bersifat private. Pada class Produk, getter dan setter diterapkan pada seluruh atribut, yaitu kode, kode_produk, nama_produk, harga, dan stock. Penggunaan method ini merupakan penerapan konsep enkapsulasi (encapsulation) dalam pemrograman berorientasi objek sehingga data tetap terlindungi dan hanya dapat diakses melalui method yang telah disediakan.

2.4 KategoriDAO.java

Class KategoriDao merupakan class yang menerapkan pola Data Access Object (DAO). Class ini berfungsi sebagai penghubung antara aplikasi dengan database untuk mengelola data kategori. Seluruh operasi seperti menampilkan, menambahkan, mengubah, menghapus, dan mengambil data kategori dilakukan melalui class ini sehingga proses akses database terpisah dari tampilan aplikasi.

```

24     public class KategoriDao {
25
26         Connection conn = DBConnection.getKoneksi();
27
28         // LOAD DATA
29         public void loadData(JTable table) {
30
31             DefaultTableModel model = new DefaultTableModel();
32             model.addColumn("kode");
33             model.addColumn("kategori");
34             model.addColumn("brand");
35
36             try {
37                 String sql = "SELECT p.kode"
38                     + "k.kategori, k.brand "
39                     + "FROM produk p "
40                     + "JOIN kategori k ON p.kode_produk = k.kode";
41
42                 PreparedStatement ps = conn.prepareStatement(sql);
43                 ResultSet rs = ps.executeQuery();
44
45                 while (rs.next()) {
46                     model.addRow(new Object[]{
47                         rs.getInt("kode"),
48                         rs.getString("kategori"),
49                         rs.getString("brand")
50                     });
51             }
52         }
53     }

```

Baris ke 26 source code di atas digunakan untuk membuat koneksi ke database dengan memanggil method getKoneksi() dari class DBConnection. Koneksi ini

digunakan oleh seluruh method pada KategoriDao untuk menjalankan proses pengolahan data.

```
53         table.setModel(model);
54     } catch (Exception e) {
55         JOptionPane.showMessageDialog(null, e.getMessage());
56     }
57 }
58 }
```

```
60 public void insert(Kategori k){
61
62     try{
63
64         // Mengecek apakah kombinasi kategori dan brand sudah ada
65         String cek =
66             "SELECT * FROM kategori WHERE kategori=? AND brand=?";
67
68         PreparedStatement psCek =
69             conn.prepareStatement(cek);
70
71         psCek.setString(1, k.getKategori());
72         psCek.setString(2, k.getBrand());
73
74         ResultSet rs = psCek.executeQuery();
75
76         if(rs.next()){
77
78             JOptionPane.showMessageDialog(null,
79                 "Tidak boleh ada kombinasi kategori dan brand yang sama.");
80
81             return;
82         }
83     }
```

```
84         String sql =
85             "INSERT INTO kategori(kategori,brand) VALUES(?,?)";
86
87         PreparedStatement ps =
88             conn.prepareStatement(sql);
89
90         ps.setString(1,k.getKategori());
91         ps.setString(2,k.getBrand());
92
93         ps.executeUpdate();
94
95         JOptionPane.showMessageDialog(null,
96             "Data berhasil disimpan.");
97     } catch (Exception e) {
98
99         JOptionPane.showMessageDialog(null,e.getMessage());
100     }
101 }
102 }
```

Method insert() digunakan untuk menyimpan data kategori baru ke dalam database. Sebelum proses penyimpanan dilakukan, sistem terlebih dahulu menjalankan query SELECT untuk memeriksa apakah kombinasi kategori dan brand sudah tersedia pada tabel kategori. Proses pengecekan dilakukan menggunakan PreparedStatement dengan mengambil nilai kategori dan brand dari objek Kategori. Jika data yang sama ditemukan, sistem akan menampilkan pesan bahwa kombinasi kategori dan brand tidak boleh sama, kemudian proses penyimpanan dihentikan menggunakan perintah return. Apabila data belum ada, sistem menjalankan query INSERT untuk menyimpan data kategori dan brand ke dalam database. Setelah

proses berhasil, aplikasi akan menampilkan pesan bahwa data berhasil disimpan. Seluruh proses penyimpanan dibungkus dalam blok try-catch untuk menangani kemungkinan terjadinya kesalahan saat mengakses database.

```
104 public void delete(int kode) {
105
106     try{
107
108         String cek =
109             "SELECT COUNT(*) jumlah FROM produk WHERE kode_produk=?";
110
111         PreparedStatement psCek = conn.prepareStatement(cek);
112         psCek.setInt(1, kode);
113
114         ResultSet rs = psCek.executeQuery();
115
116         if (rs.next() && rs.getInt("jumlah") > 0){
117
118             JOptionPane.showMessageDialog(null,
119                 "Kategori tidak dapat dihapus karena masih ada kategori yang berelasi");
120
121             return;
122         }
123     }
```

```
124         String sql =
125             "DELETE FROM kategori WHERE kode=?";
126
127         PreparedStatement ps =
128             conn.prepareStatement(sql);
129
130         ps.setInt(1, kode);
131
132         ps.executeUpdate();
133
134         JOptionPane.showMessageDialog(null,
135             "Data berhasil dihapus.");
136     } catch (Exception e){
137
138         JOptionPane.showMessageDialog(null,
139             e.getMessage());
140     }
141 }
142 }
```

Method delete() digunakan untuk menghapus data kategori berdasarkan kode kategori yang dipilih. Sebelum proses penghapusan dilakukan, sistem terlebih dahulu menjalankan query Select Count(*) untuk memeriksa apakah kategori tersebut masih digunakan pada tabel produk. Proses pengecekan dilakukan menggunakan PreparedStatement, kemudian hasilnya disimpan ke dalam ResultSet. Apabila jumlah data yang berelasi lebih dari nol, sistem akan menampilkan pesan bahwa kategori tidak dapat dihapus karena masih memiliki relasi dengan data produk, sehingga proses penghapusan dibatalkan menggunakan perintah return. Jika kategori tidak memiliki relasi dengan tabel produk, sistem menjalankan query Delete untuk menghapus data berdasarkan kode kategori. Setelah data berhasil dihapus, aplikasi akan menampilkan pesan bahwa data berhasil dihapus. Seluruh

proses dibungkus dalam blok try-catch untuk menangani kemungkinan terjadinya kesalahan saat mengakses database.

```
144 public void update(Kategori k) {
145
146     try {
147
148         String sql = "INSERT INTO kategori(kategori,brand) VALUES(?,?)";
149
150         PreparedStatement ps = conn.prepareStatement(sql);
151
152         ps.setString(1, k.getKategori());
153         ps.setString(2, k.getBrand());
154
155         ps.executeUpdate();
156
157     } catch (Exception e) {
158         JOptionPane.showMessageDialog(null, e.getMessage());
159     }
160 }
```

Method update() digunakan untuk memperbarui data kategori yang telah tersimpan di dalam database. Method ini menerima objek Kategori sebagai parameter yang berisi data terbaru, yaitu kategori, brand, dan kode. Selanjutnya sistem menjalankan query UPDATE menggunakan PreparedStatement untuk mengubah data pada tabel kategori berdasarkan kode kategori yang dipilih. Nilai kategori dan brand yang baru akan menggantikan data sebelumnya, sedangkan kode digunakan sebagai acuan untuk menentukan data yang akan diperbarui. Setelah query berhasil dijalankan menggunakan executeUpdate(), sistem akan memeriksa hasil proses pembaruan. Jika data berhasil diperbarui, aplikasi akan menampilkan pesan "Data berhasil diupdate." kepada pengguna. Seluruh proses pembaruan data dibungkus dalam blok try-catch untuk menangani kemungkinan terjadinya kesalahan saat mengakses database.

```
162 public List<Kategori> getAllKategori() {
163     List<Kategori> list = new ArrayList<>();
164
165     try {
166
167         String sql = "SELECT * FROM kategori";
168         PreparedStatement ps = conn.prepareStatement(sql);
169         ResultSet rs = ps.executeQuery();
170
171         while (rs.next()) {
172
173             Kategori k = new Kategori();
174
175             k.setKode(rs.getInt("kode"));
176             k.setKategori(rs.getString("kategori"));
177             k.setBrand(rs.getString("brand"));
178
179             list.add(k);
180         }
181
182     } catch (Exception e) {
183         JOptionPane.showMessageDialog(null, e.getMessage());
184     }
185
186     return list;
187 }
```

Method `getAllKategori()` digunakan untuk mengambil seluruh data kategori yang terdapat pada database. Pada awal method dibuat sebuah objek `List<Kategori>` menggunakan `ArrayList` sebagai tempat penyimpanan data kategori yang diperoleh dari database. Selanjutnya sistem menjalankan query `SELECT * FROM kategori` menggunakan `PreparedStatement`, kemudian hasilnya disimpan ke dalam `ResultSet`. Data pada `ResultSet` dibaca satu per satu menggunakan perulangan `while`, kemudian setiap data dimasukkan ke dalam objek `Kategori` melalui method setter. Objek `Kategori` yang telah berisi data selanjutnya ditambahkan ke dalam `List` menggunakan `list.add(k)`. Setelah seluruh data selesai diproses, method mengembalikan daftar kategori menggunakan `return list`, sehingga data tersebut dapat digunakan oleh form lain, misalnya untuk mengisi `JComboBox` atau menampilkan daftar kategori pada aplikasi.

```
189 public int getCodeKategori(String namaKategori) {
190
191     int kode = 0;
192
193     try {
194
195         String sql = "SELECT kode FROM kategori WHERE kategori = ?";
196
197         PreparedStatement ps = conn.prepareStatement(sql);
198         ps.setString(1, namaKategori);
199
200         ResultSet rs = ps.executeQuery();
201
202         if (rs.next()) {
203             kode = rs.getInt("kode");
204         }
205
206     } catch (Exception e) {
207         JOptionPane.showMessageDialog(null, e.getMessage());
208     }
209
210     return kode;
211 }
```

Method `getCodeKategori()` digunakan untuk mengambil kode kategori berdasarkan nama kategori yang dipilih oleh pengguna. Method ini menerima parameter berupa nama kategori, kemudian menjalankan query `Select kode From kategori Where kategori = ?` menggunakan `PreparedStatement`. Hasil query disimpan ke dalam `ResultSet`, kemudian sistem memeriksa apakah data ditemukan menggunakan `rs.next()`. Jika data tersedia, nilai kode kategori akan diambil dan disimpan ke dalam variabel `kode`. Apabila terjadi kesalahan saat proses pengambilan data, sistem akan menampilkan pesan kesalahan menggunakan `JOptionPane`. Pada akhir method, nilai

kode kategori dikembalikan menggunakan return kode sehingga dapat digunakan oleh class lain, misalnya saat proses penyimpanan data produk.

```
212 public boolean cekKategoriBrand(String kategori, String brand) {
213
214     try {
215
216         String sql = "SELECT * FROM kategori "
217             + "WHERE kategori=? AND brand=?";
218
219         PreparedStatement ps = conn.prepareStatement(sql);
220
221         ps.setString(1, kategori);
222         ps.setString(2, brand);
223
224         ResultSet rs = ps.executeQuery();
225
226         return rs.next();
227
228     } catch (Exception e) {
229         JOptionPane.showMessageDialog(null, e.getMessage());
230
231         return false;
232     }
233 }
234
235 }
```

Method `cekKategoriBrand()` digunakan untuk memeriksa apakah kombinasi kategori dan brand sudah terdapat di dalam database. Method ini menerima dua parameter, yaitu kategori dan brand, kemudian menjalankan query Select form kategori where kategori=? And brand=? menggunakan PreparedStatement. Nilai parameter kategori dan brand dimasukkan ke dalam query menggunakan method `setString()`, kemudian query dijalankan menggunakan `executeQuery()`. Hasil query disimpan pada objek `ResultSet`, selanjutnya method mengembalikan nilai true apabila data ditemukan (`rs.next()` bernilai benar) dan false apabila data tidak ditemukan atau terjadi kesalahan saat proses pengecekan. Method ini digunakan sebagai validasi untuk mencegah penyimpanan data kategori dengan kombinasi kategori dan brand yang sama sehingga menjaga konsistensi data pada database.

2.5 ProdukDAO.java

Class `ProdukDao` merupakan class yang menerapkan pola Data Access Object (DAO) untuk mengelola data produk pada database. Class ini berfungsi sebagai penghubung antara aplikasi dengan database dalam melakukan proses menampilkan, menambahkan, mengubah, dan menghapus data produk. Dengan

adanya class ini, seluruh proses akses database dipisahkan dari tampilan aplikasi sehingga kode program menjadi lebih terstruktur dan mudah dipelihara.

```
23 public class ProdukDao {
24
25     Connection conn = DBConnection.getKoneksi();
26
27     // LOAD DATA
28     public void loadData(JTable table) {
29
30         DefaultTableModel model = new DefaultTableModel();
31
32         model.addColumn("Kode");
33         model.addColumn("Kode Produk"); // FK (akan disembunyikan)
34         model.addColumn("Nama Produk");
35         model.addColumn("Harga");
36         model.addColumn("Stock");
37         model.addColumn("Kategori");
38         model.addColumn("Brand");
39
40         try {
41
42             String sql = "SELECT p.kode, p.kode_produk, p.nama_produk, p.harga, p.stock, "
43                 + "k.kategori, k.brand "
44                 + "FROM produk p "
45                 + "INNER JOIN kategori k "
46                 + "ON p.kode_produk = k.kode";
47
48             PreparedStatement ps = conn.prepareStatement(sql);
49             ResultSet rs = ps.executeQuery();
```

Baris ke 25 source code di atas digunakan untuk membuat koneksi ke database dengan memanggil method `getKoneksi()` dari class `DBConnection`. Koneksi ini digunakan oleh seluruh method pada class `ProdukDao` untuk menjalankan proses pengolahan data produk.

Method `loadData()` digunakan untuk mengambil seluruh data produk dari database dan menampilkannya ke dalam komponen `JTable`. Pada awal method dibuat sebuah `DefaultTableModel` beserta kolom-kolom yang akan ditampilkan pada tabel, yaitu kode, kode produk, nama produk, harga, stok, kategori, dan brand. Selanjutnya sistem menjalankan query `Select` yang menggunakan `Inner Join` untuk menggabungkan data dari tabel produk dan kategori, sehingga informasi kategori dan brand dapat ditampilkan bersama data produk. Hasil query disimpan ke dalam `ResultSet`, kemudian setiap data dibaca menggunakan perulangan `while` dan ditambahkan ke dalam tabel menggunakan `model.addRow()`. Setelah seluruh data berhasil dimuat, model ditampilkan pada `JTable` menggunakan `table.setModel(model)`. Pada bagian akhir method, kolom `kode_produk` disembunyikan karena hanya digunakan sebagai `Foreign Key` dan tidak perlu ditampilkan kepada pengguna.


```

51 while (rs.next()) {
52
53     model.addRow(new Object[]{
54         rs.getInt("kode"),
55         rs.getInt("kode_produk"),
56         rs.getString("nama_produk"),
57         rs.getInt("harga"),
58         rs.getInt("stock"),
59         rs.getString("kategori"),
60         rs.getString("brand")
61     });
62
63 }
64
65 table.setModel(model);
66
67 // sembunyikan kolom kode_produk
68 table.getColumnModel().getColumn(1).setMinWidth(0);
69 table.getColumnModel().getColumn(1).setMaxWidth(0);
70 table.getColumnModel().getColumn(1).setWidth(0);
71
72 } catch (Exception e) {
73     JOptionPane.showMessageDialog(null, e.getMessage());
74 }
75 }

```

```

77 // INSERT
78 public void insert(Produk p) {
79
80     try {
81         String sql = "INSERT INTO produk(kode_produk,nama_produk,harga,stock) VALUES(?,?,?,?)";
82
83         PreparedStatement ps = conn.prepareStatement(sql);
84         ps.setInt(1, p.getKodeProduk());
85         ps.setString(2, p.getNama());
86         ps.setInt(3, p.getHarga());
87         ps.setInt(4, p.getStock());
88
89         ps.executeUpdate();
90
91         JOptionPane.showMessageDialog(null, "Data berhasil disimpan");
92
93     } catch (Exception e) {
94         JOptionPane.showMessageDialog(null, e.getMessage());
95     }
96 }

```

Method insert() digunakan untuk menyimpan data produk baru ke dalam database. Method ini menerima objek Produk sebagai parameter yang berisi data produk yang akan disimpan. Selanjutnya sistem menjalankan query Insert Into produk menggunakan PreparedStatement, kemudian nilai kode produk, nama produk, harga, dan stok diambil dari objek Produk melalui method getter. Setelah proses penyimpanan berhasil dilakukan menggunakan executeUpdate(), aplikasi akan menampilkan pesan bahwa data berhasil disimpan. Seluruh proses dibungkus dalam blok try-catch untuk menangani kemungkinan terjadinya kesalahan saat mengakses database.


```

98 // UPDATE
99 public void update(Produk p) {
100
101     try {
102
103         String sql = "UPDATE produk SET kode_produk=?, nama_produk=?, harga=?, stock=? WHERE kode=?";
104
105         PreparedStatement ps = conn.prepareStatement(sql);
106
107         ps.setInt(1, p.getKodeProduk());
108         ps.setString(2, p.getNama());
109         ps.setInt(3, p.getHarga());
110         ps.setInt(4, p.getStock());
111         ps.setInt(5, p.getKode());
112
113         ps.executeUpdate();
114
115         JOptionPane.showMessageDialog(null, "Data berhasil diupdate");
116
117     } catch (Exception e) {
118         JOptionPane.showMessageDialog(null, e.getMessage());
119     }
120 }

```

Method `update()` digunakan untuk memperbarui data produk yang telah tersimpan di dalam database. Method ini menerima objek `Produk` sebagai parameter yang berisi data terbaru. Selanjutnya sistem menjalankan query Update menggunakan `PreparedStatement` untuk mengubah data produk berdasarkan kode yang dipilih. Setelah proses pembaruan berhasil dilakukan menggunakan `executeUpdate()`, aplikasi akan menampilkan pesan bahwa data berhasil diperbarui. Apabila terjadi kesalahan selama proses pembaruan data, sistem akan menampilkan pesan kesalahan melalui `JOptionPane`.

```

122 // DELETE
123 public void delete(int kode) {
124
125     try {
126
127         String sql = "DELETE FROM produk WHERE kode=?";
128
129         PreparedStatement ps = conn.prepareStatement(sql);
130         ps.setInt(1, kode);
131
132         ps.executeUpdate();
133
134         JOptionPane.showMessageDialog(null, "Data berhasil dihapus");
135
136     } catch (Exception e) {
137         JOptionPane.showMessageDialog(null, e.getMessage());
138     }
139 }
140 }

```

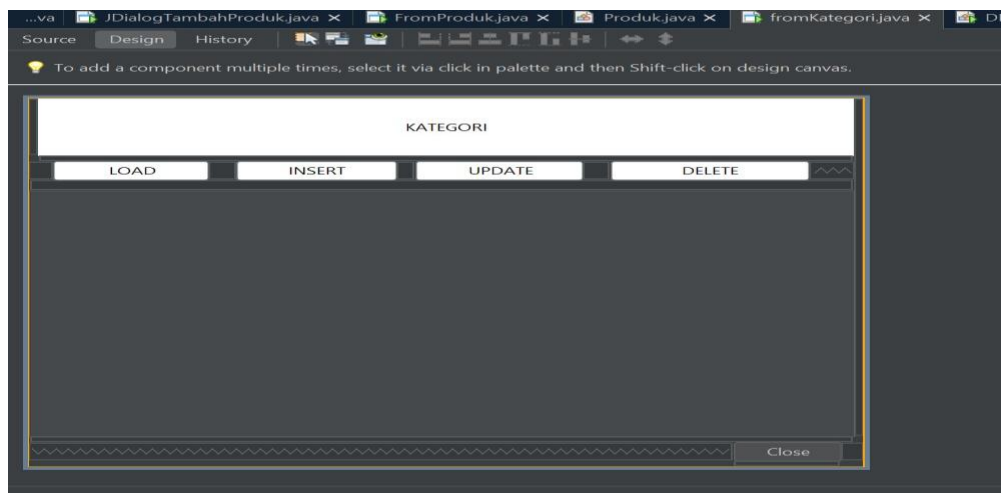
Method `delete()` digunakan untuk menghapus data produk dari database berdasarkan kode produk yang dipilih. Sistem menjalankan query Delete menggunakan `PreparedStatement`, kemudian nilai kode dikirim sebagai parameter untuk menentukan data yang akan dihapus. Setelah proses penghapusan berhasil dilakukan menggunakan `executeUpdate()`, aplikasi akan menampilkan pesan bahwa

data berhasil dihapus. Seluruh proses penghapusan dibungkus dalam blok try-catch untuk menangani kemungkinan terjadinya kesalahan saat mengakses database.

3. Implementasi Aplikasi

3.1 Form Kategori

Form Kategori digunakan untuk mengelola data kategori pada aplikasi. Melalui form ini, pengguna dapat menampilkan, menambahkan, mengubah, dan menghapus data kategori. Data yang dikelola meliputi kategori dan brand, kemudian ditampilkan dalam bentuk tabel.



Penjelasan Tombol :

a. Tombol Load

Tombol Load digunakan untuk menampilkan seluruh data kategori yang telah tersimpan pada database ke dalam tabel (JTable). Dengan menekan tombol ini, pengguna dapat melihat data kategori beserta brand yang tersedia.

b. Tombol Insert

Tombol Insert digunakan untuk membuka JDialog Tambah Kategori. Melalui dialog tersebut, pengguna dapat memasukkan data kategori dan brand baru sebelum disimpan ke dalam database.

c. Tombol Update

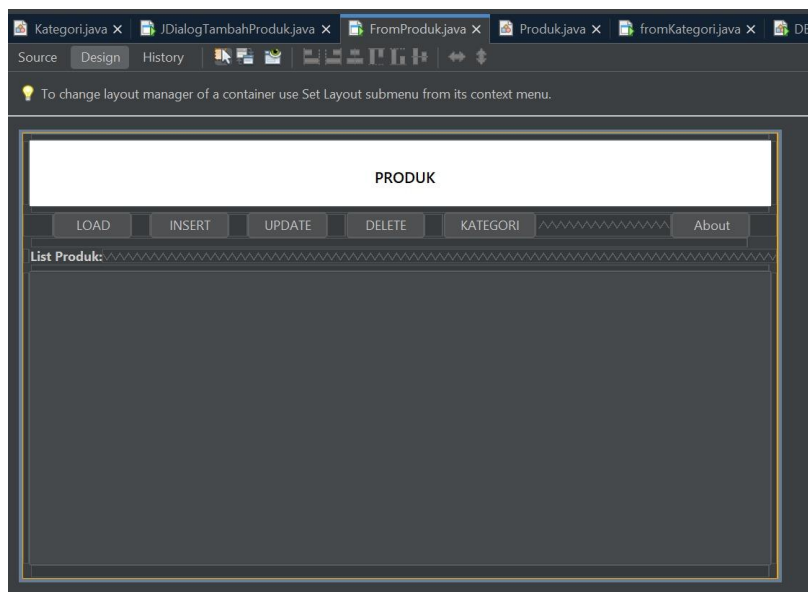
Tombol Update digunakan untuk mengubah data kategori yang telah dipilih pada tabel. Setelah pengguna memilih salah satu data, aplikasi akan menampilkan dialog yang berisi data lama sehingga pengguna dapat melakukan perubahan, kemudian menyimpan hasil perubahan tersebut.

d. Tombol Delete

Tombol Delete digunakan untuk menghapus data kategori yang dipilih dari database. Sebelum proses penghapusan dilakukan, sistem akan memeriksa apakah data kategori masih memiliki relasi dengan data produk. Jika masih terdapat relasi, maka data tidak dapat dihapus dan aplikasi akan menampilkan pesan peringatan.

3.2 Form Produk

Form Produk digunakan untuk mengelola data produk pada aplikasi Sistem Kasir. Melalui form ini, pengguna dapat menampilkan, menambahkan, mengubah, dan menghapus data produk. Selain itu, form ini juga menyediakan akses ke Form Kategori untuk mengelola data kategori yang akan digunakan pada produk, serta tombol About untuk menampilkan informasi mengenai aplikasi.



Penjelasan Tombol :

a. Tombol Load

Tombol Load digunakan untuk menampilkan seluruh data produk yang tersimpan di dalam database ke dalam tabel (JTable). Data yang ditampilkan meliputi nama produk, harga, stok, kategori, dan brand.

b. Tombol Insert

Tombol Insert digunakan untuk membuka JDialog Tambah Produk. Pada dialog tersebut, pengguna dapat mengisi data produk seperti kategori, nama produk, harga, dan stok sebelum data disimpan ke dalam database.

c. Tombol Update

Tombol Update digunakan untuk mengubah data produk yang dipilih pada tabel. Setelah pengguna memilih salah satu data, aplikasi akan menampilkan dialog yang berisi data produk sebelumnya sehingga pengguna dapat melakukan perubahan dan menyimpan hasil pembaruan ke database.

d. Tombol Delete

Tombol Delete digunakan untuk menghapus data produk yang dipilih dari database. Setelah proses penghapusan berhasil dilakukan, data pada tabel akan diperbarui sehingga pengguna dapat langsung melihat perubahan.

e. Tombol Kategori

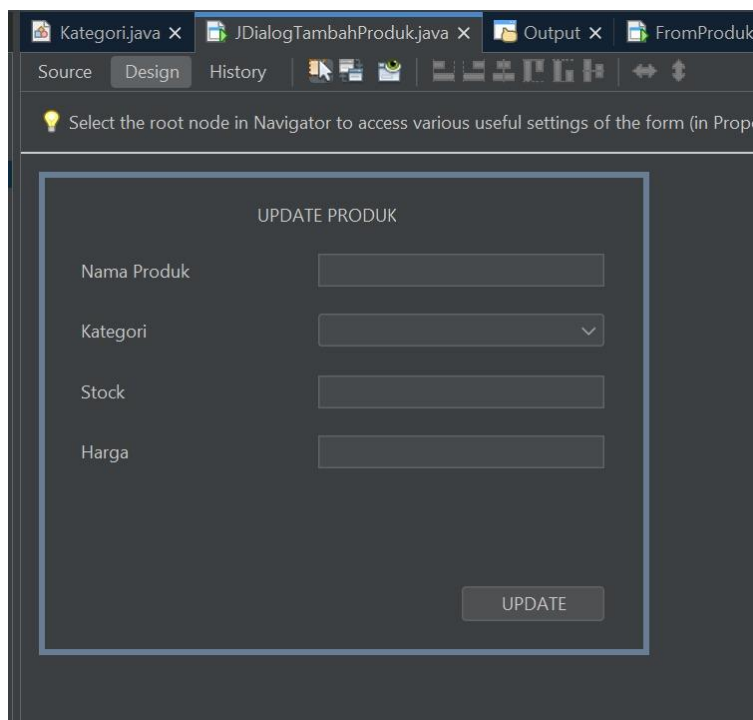
Tombol Kategori digunakan untuk membuka Form Kategori. Melalui form tersebut, pengguna dapat mengelola data kategori dan brand yang nantinya digunakan sebagai referensi saat menambahkan data produk.

f. Tombol About

Tombol About digunakan untuk menampilkan informasi mengenai aplikasi, seperti nama aplikasi, identitas pembuat, atau informasi lain yang berkaitan dengan sistem.

3.3 Jdialog produk

JDialog Update Produk digunakan untuk mengubah data produk yang telah tersimpan di dalam database. Dialog ini akan ditampilkan setelah pengguna memilih salah satu data pada tabel kemudian menekan tombol Update pada Form Produk. Data produk yang dipilih akan ditampilkan secara otomatis sehingga pengguna dapat melakukan perubahan sesuai kebutuhan.



Penjelasan Komponen :

- a. TextField Nama Produk : Komponen Nama Produk digunakan untuk memasukkan atau mengubah nama produk yang dipilih.
- b. ComboBox Kategori : Komponen Kategori digunakan untuk memilih kategori produk. Data kategori yang ditampilkan pada ComboBox diambil dari tabel kategori sehingga pengguna dapat memilih kategori yang telah tersedia.

- c. TextField Stock : Komponen Stock digunakan untuk memasukkan atau mengubah jumlah stok produk yang tersedia.
- d. TextField Harga : Komponen Harga digunakan untuk memasukkan atau mengubah harga produk sesuai dengan nilai yang diinginkan.
- e. Tombol Update : Tombol Update digunakan untuk menyimpan perubahan data produk yang telah dilakukan oleh pengguna. Setelah tombol ditekan, aplikasi akan melakukan validasi data kemudian memperbarui data produk pada database. Jika proses berhasil, aplikasi akan menampilkan pesan bahwa data berhasil diperbarui dan tabel pada Form Produk akan menampilkan data terbaru.

4. Pembahasan

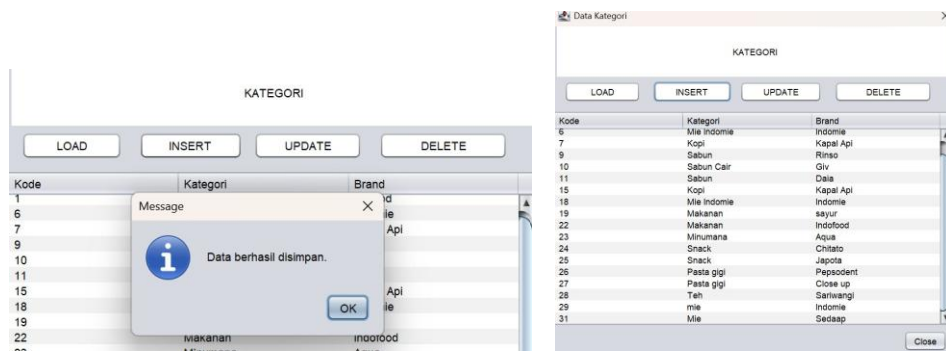
4.1 Cara Menjalankan Aplikasi

a. Menampilkan Data Produk

Kode	Nama Produk	Harga	Stock	Kategori	Brand
9	Sayang	5000	5	Sabun	Rinso
16	kopikap	1000	6	Kopi	Kapal Api
19	Barber Daily Acne Control 250ml	35000	16	Sabun Cair	Giv
30	Lifeboy	22000	5	Sabun	Rinso
43	Kopi Kapal Api 15 ml	10000	20	Kopi	Kapal Api
44	Chotato rasa Rumput Jepang	5000	4	Snack	Chitato
45	Japota rasa Madu	11000	8	Snack	Japota
46	Sariwangi Teh Melati	7000	12	Teh	Sariwangi
47	Pepsodent 12 ml	19000	35	Pasta gigi	Pepsodent
48	Sabun Nuvo 5 ml	5000	10	Sabun Cair	Giv
49	Sayur Tempe	10000	2	Makanan	sayur
50	Mie Korean Spicy	3400	14	mie	Indomie
51	Aqua Tanggung	6000	6	Minumana	Aqua

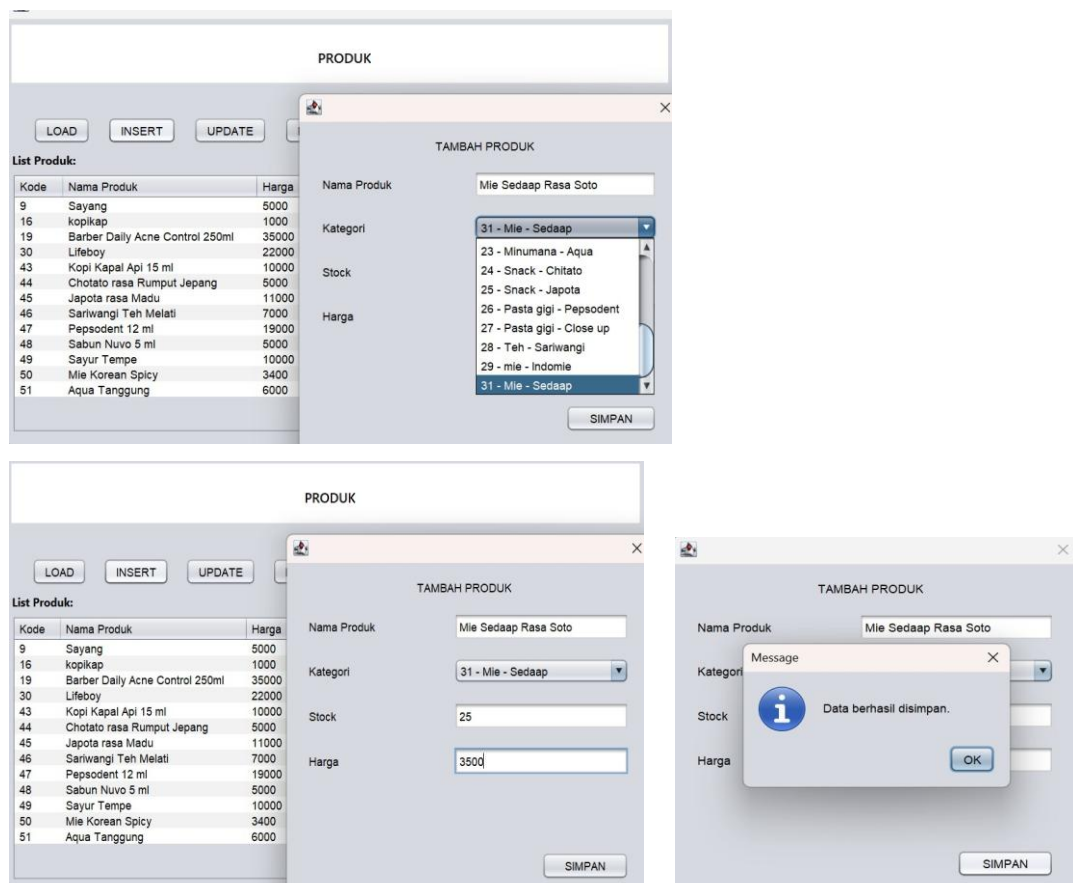
b. Menambahkan Data Kategori

Kode	Kategori	Brand
1		
6		
7		
9		
10		
11		
15		
18		
19		
22	Makanan	Indomie
23	Minumana	Aqua
24	Snack	Chitato
25	Snack	Japota
26	Pasta gigi	Pepsodent
27	Pasta gigi	Close up
28	Teh	Sariwangi
29	mie	Indomie



Sebelum menambahkan produk baru, pengguna dapat membuka Form Kategori melalui tombol Kategori. Selanjutnya tekan tombol Insert, masukkan nama kategori dan brand, kemudian simpan data. Jika proses berhasil, kategori baru akan tersimpan di database dan muncul pada tabel kategori.

c. Menambahkan Data Produk



Pengguna menekan tombol Insert pada Form Produk untuk membuka dialog tambah produk. Selanjutnya pengguna memilih kategori, mengisi nama produk,

harga, dan stok. Setelah tombol Simpan ditekan, data akan disimpan ke database dan ditampilkan pada tabel produk.

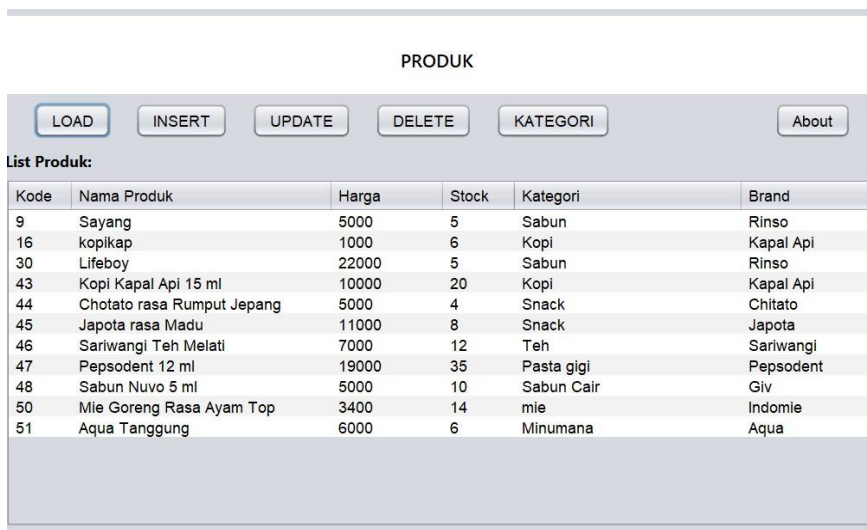
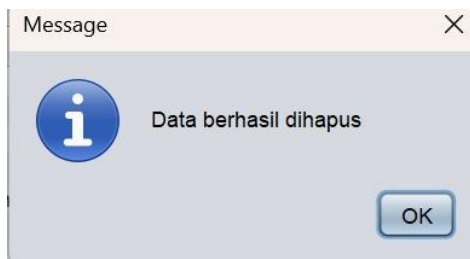
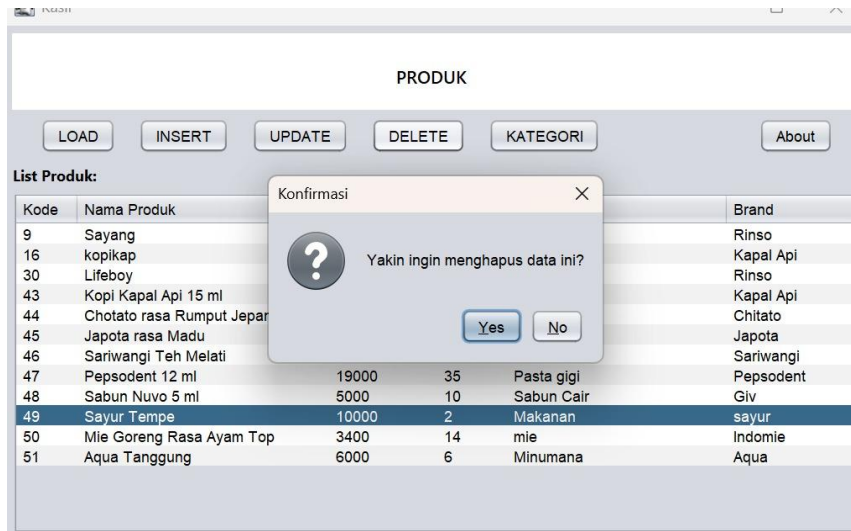
d. Mengubah Data Produk

The image shows two screenshots of a software application. The top-left screenshot shows the 'UPDATE PRODUK' form with the following fields: Nama Produk (Mie Korean Spicy), Kategori (29 - mie - Indomie), Stock (14), and Harga (3400). The top-right screenshot shows the same form with the 'UPDATE' button pressed, and a 'Message' dialog box appears with the text 'Data berhasil diupdate' and an 'OK' button. The bottom screenshot shows the 'PRODUK' table with the following data:

Kode	Nama Produk	Harga	Stock	Kategori	Brand
9	Sayang	5000	5	Sabun	Rinso
16	kopikap	1000	6	Kopi	Kapal Api
19	Barber Daily Acne Control 250ml	35000	16	Sabun Cair	Giv
30	Lifeboy	22000	5	Sabun	Rinso
43	Kopi Kapal Api 15 ml	10000	20	Kopi	Kapal Api
44	Chotato rasa Rumput Jepang	5000	4	Snack	Chitato
45	Japota rasa Madu	11000	8	Snack	Japota
46	Sariwangi Teh Melati	7000	12	Teh	Sariwangi
47	Pepsodent 12 ml	19000	35	Pasta gigi	Pepsodent
48	Sabun Nuvo 5 ml	5000	10	Sabun Cair	Giv
49	Sayur Tempe	10000	2	Makanan	sayur
50	Mie Goreng Rasa Ayam Top	3400	14	mie	Indomie
51	Aqua Tanggung	6000	6	Minumana	Aqua
52	Mie Sedaap Rasa Soto	3500	25	Mie	Sedaap

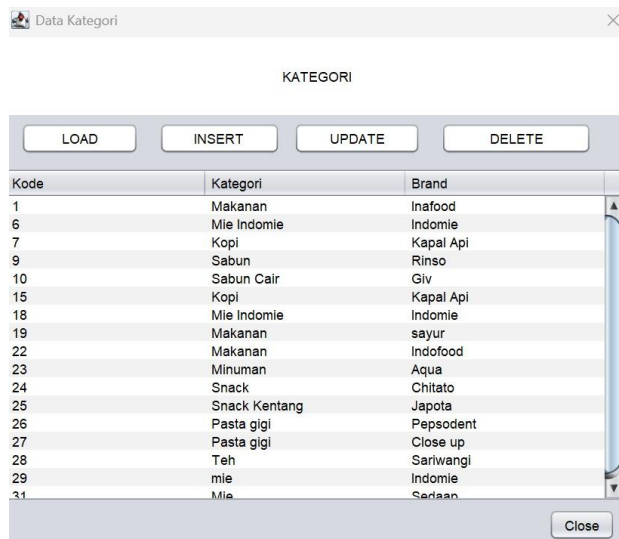
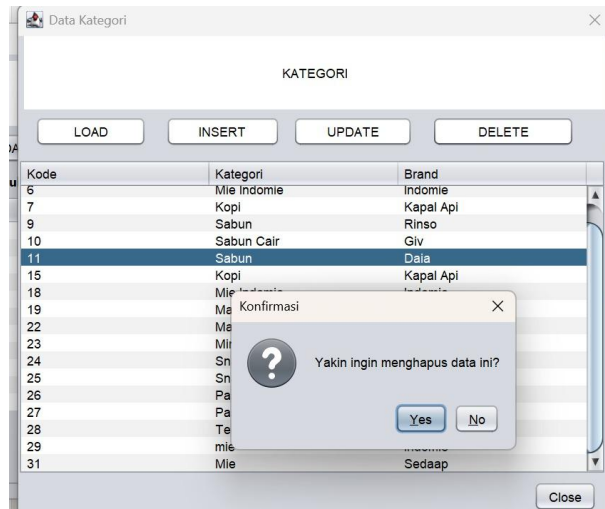
Untuk mengubah data produk, pengguna memilih salah satu data pada tabel kemudian menekan tombol Update. Setelah data diperbarui dan tombol Update ditekan, sistem akan menyimpan perubahan ke database dan menampilkan pesan bahwa data berhasil diperbarui.

e. Menghapus Data Produk



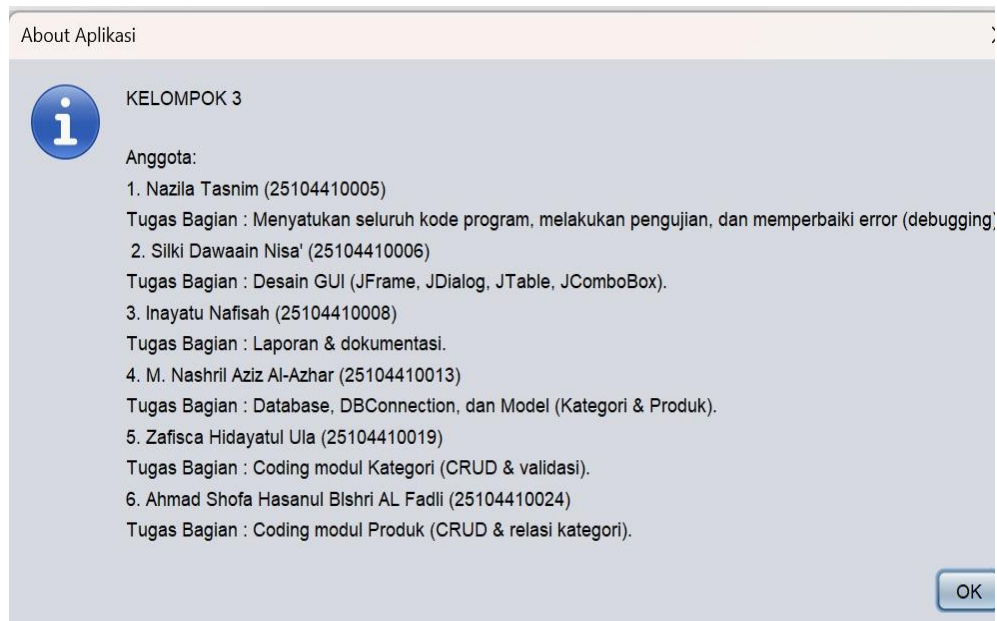
Pengguna memilih data yang akan dihapus kemudian menekan tombol Delete. Setelah memilih Yes pada kotak konfirmasi, sistem akan menghapus data dari database dan menampilkan pesan bahwa data berhasil dihapus.

f. Menghapus Data Kategori



Saat menghapus kategori, sistem akan memeriksa apakah kategori tersebut masih digunakan oleh data pada tabel produk. Jika masih memiliki relasi, maka penghapusan dibatalkan dan aplikasi menampilkan pesan peringatan. Apabila tidak terdapat relasi, data kategori akan berhasil dihapus dari database.

- g. Klik About untuk melihat informasi aplikasi



4.2 Penggunaan Pola DAO (Data Access Object)

Pada aplikasi ini digunakan pola Data Access Object (DAO) untuk memisahkan proses pengolahan data dengan tampilan aplikasi. Seluruh proses yang berhubungan dengan database ditempatkan pada class KategoriDao dan ProdukDao, sedangkan form hanya bertugas menerima input dari pengguna dan menampilkan hasilnya. Dengan demikian, source code menjadi lebih terstruktur, mudah dipelihara, dan lebih mudah dikembangkan.

- a. Inisialisasi Koneksi Database

```
Connection conn = DBConection.getKoneksi();
```

Baris source code di atas digunakan untuk membuat koneksi ke database dengan memanggil method `getKoneksi()` dari class `DBConection`. Koneksi tersebut disimpan pada variabel `conn` dan digunakan oleh seluruh method pada class DAO untuk menjalankan perintah SQL.

b. Method insert()

```
77 // INSERT
78 public void insert(Produk p) {
79
80     try {
81         String sql = "INSERT INTO produk(kode_produk,nama_produk,harga,stock) VALUES(?,?,?,?)";
82
83         PreparedStatement ps = conn.prepareStatement(sql);
84         ps.setInt(1, p.getKodeProduk());
85         ps.setString(2, p.getNama());
86         ps.setInt(3, p.getHarga());
87         ps.setInt(4, p.getStock());
88
89         ps.executeUpdate();
90
91         JOptionPane.showMessageDialog(null, "Data berhasil disimpan");
92
93     } catch (Exception e) {
94         JOptionPane.showMessageDialog(null, e.getMessage());
95     }
96 }
```

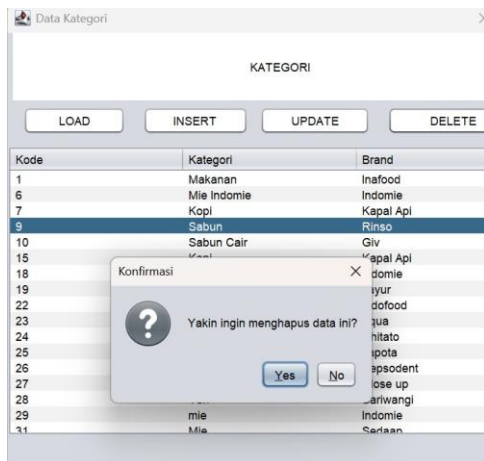
Method insert() digunakan untuk menyimpan data produk ke dalam database. Query SQL disimpan pada variabel sql, kemudian dibuat objek PreparedStatement untuk menjalankan query tersebut. Nilai parameter diambil dari objek Produk melalui method getter, kemudian disisipkan ke dalam query menggunakan setInt() dan setString(). Setelah semua parameter terisi, method executeUpdate() dijalankan untuk menyimpan data ke database. Apabila terjadi kesalahan selama proses penyimpanan, sistem akan menampilkan pesan kesalahan melalui JOptionPane.

c. Pemanggilan DAO dari Form

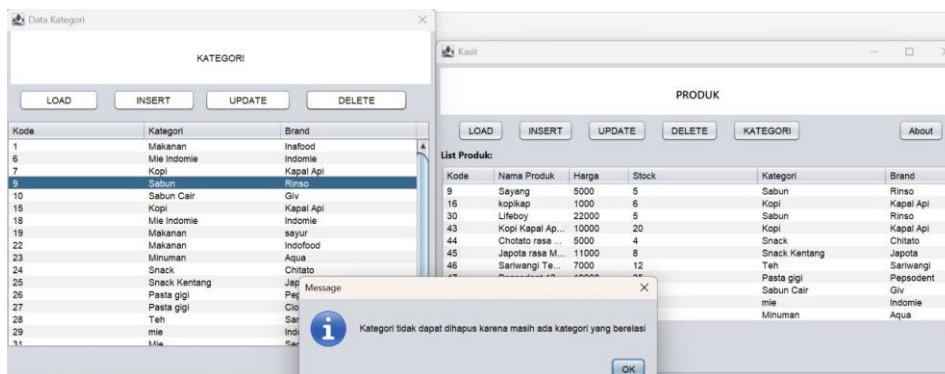
```
31 private void tampilData() {
32
33     ProdukDao dao = new ProdukDao();
34     dao.loadData(jTable1);
35     TableColumnModel kolom = jTable1.getColumnModel();
36     kolom.getColumn(0).setPreferredWidth(50); // Kode
37     kolom.getColumn(2).setPreferredWidth(80); // Harga
38     kolom.getColumn(3).setPreferredWidth(60); // Stock
39     kolom.getColumn(4).setPreferredWidth(150); // Kategori
40     kolom.getColumn(5).setPreferredWidth(150); // Brand
41 }
```

Potongan source code di atas menunjukkan bahwa form hanya memanggil method yang terdapat pada class DAO. Dengan demikian, proses akses database dipisahkan dari tampilan aplikasi sesuai konsep Data Access Object (DAO).

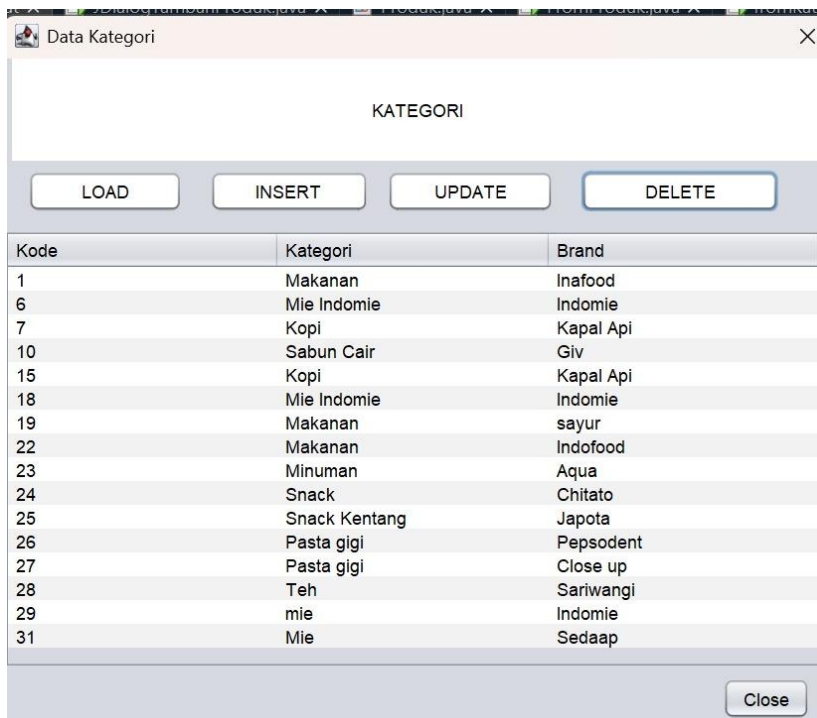
4.3 Penanganan Error Foreign Key



Sebelum data kategori dihapus, aplikasi akan menampilkan kotak dialog konfirmasi untuk memastikan bahwa pengguna benar-benar ingin menghapus data tersebut. Jika pengguna memilih tombol Yes, maka sistem akan melanjutkan proses pengecekan relasi data pada database. Sebaliknya, jika pengguna memilih tombol No, maka proses penghapusan akan dibatalkan.



Setelah pengguna mengonfirmasi proses penghapusan, aplikasi akan memeriksa apakah kategori tersebut masih digunakan oleh data pada tabel produk. Proses pengecekan dilakukan menggunakan query `SELECT COUNT(*)` pada tabel produk berdasarkan kode kategori. Apabila hasil pengecekan menunjukkan bahwa kategori masih memiliki relasi dengan data produk, maka aplikasi akan membatalkan proses penghapusan dan menampilkan pesan "Kategori tidak dapat dihapus karena masih ada kategori yang berelasi." Mekanisme ini diterapkan untuk menjaga konsistensi data serta mencegah terjadinya pelanggaran Foreign Key Constraint pada database.



Kode	Kategori	Brand
1	Makanan	Inafood
6	Mie Indomie	Indomie
7	Kopi	Kapal Api
10	Sabun Cair	Giv
15	Kopi	Kapal Api
18	Mie Indomie	Indomie
19	Makanan	sayur
22	Makanan	Indofood
23	Minuman	Aqua
24	Snack	Chitato
25	Snack Kentang	Japota
26	Pasta gigi	Pepsodent
27	Pasta gigi	Close up
28	Teh	Sariwangi
29	mie	Indomie
31	Mie	Sedaap

Jika kategori yang dipilih tidak memiliki relasi dengan data produk, maka sistem akan menghapus data tersebut dari database. Setelah proses penghapusan berhasil, aplikasi menampilkan pesan "Data berhasil dihapus" sebagai konfirmasi kepada pengguna. Data pada tabel kemudian diperbarui sehingga perubahan dapat langsung terlihat.